ICT374 Lab 6:
Text Processing in C, Tokenization of Input Lines; Unix Signals, Sending SIGINT, SIGQUIT and SIGTSTP from Keyboard, Kill a Program with SIGKILL. Process Signals Using `sigaction` structure and `sigaction` Call, Catch Signals, Block and Unblock Signals;

Learning Objectives:

1. Learn to process texts with C's string handling functions.
2. Learn to break an input line into a sequence of tokens.
3. Understand Unix signals and their default meanings.
4. Be able to send SIGINT and SIGQUIT and SIGTSTP signals to foreground processes from keyboard.
5. Be able to kill a process such as an editor using SIGKILL.
6. Be able to process reliable signals using `sigaction` structure and `sigaction` call.
7. Be able to ignore a signal and restore a signal.
8. Be able to write your own signal handlers and to setup such signal handlers to catch signals.
9. Be able to block and unblock signals.

Required Reading:

- Lecture Notes for Topic 6
- Functions in `string.h`..
- `signal.h` and other related header files for signals.
- The manual pages of signal related system calls and functions
- In addition to the lecture notes on Signal Handling, the relevant chapters in Stevens and Rago (chapter 10).

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also each exercise should be in a separate directory, which itself, is under a directory called `lab06`

1. **Getting command lines from the user**

   Write a program that repeatedly:

   a. display a prompt such as "%", "$", or whatever you like, to the terminal output;
   b. then input one line of text from the user;
   c. then prints out the input line to the terminal output if the input line is not "exit".

   Your program should quit if the input line is "exit".

You may use the function `fgets` to read an input line. Please note that `fgets` keeps the newline character in its input line. You should check whether the last character is newline (`'\n'`) and remove it from the input line if it is.

2. **Tokenizer for a text line**

   Write a module named `token` (ie, `token.h` and `token.c`). The tokenizer exports the following function

   ```
   int tokenise (char *inputLine, char *token[])
   ```

   The function takes a null-terminated line of characters `inputLine` and breaks it into a sequence of tokens and assigns the starting addresses of these tokens into the array `tokens`. The actual parameter for `tokens` should be declared to be an array of `MAX_NUM_TOKENS` number of char pointers, such as the one below:

   ```
   char *token[MAX_NUM_TOKENS];
   ```

   where `MAX_NUM_TOKEN` is a constant such as

   ```
   #define MAX_NUM_TOKENS   100
   ```

   The function should find the start address of *i*th token and assign the address to the corresponding array element `token[i]`. It must also replace the character at the end of that token with the null character `'\0'`. The function should return the total number of tokens found in the input line or -1 if the array passed to `token` is too small for the tokens (ie the number of array elements is less than the number of tokens found in the input line).

   For simplicity you can assume that any two neighbouring tokens are separated by at least one white space character, which is either space `' '`, or tab `'\t'` or newline `'\n'`.

   Note you must place the constant declaration `MAX_NUM_TOKENS` and function prototype for `tokenise` in the header file `token.h`. Place the rest in the implementation file `token.c`.

   In designing your tokenizer, you may want to use the functions listed in `string.h`. The following web page lists all functions available from `string.h`:

   http://www.cplusplus.com/ref/#libs

   As a part of this question, you should write a small main program to test function `tokenise` from module `token`.

   If you find the question difficult to do, you may start from the following code: `token.h` and `token.c`.

3. **Breaking a command line into a sequence of tokens**

   Modify your program from C Programming Exercise 1 above, so that instead of printing the input line it reads, it breaks up the input line into a sequence of tokens and prints out the list of tokens. You should use module `token` from Exercise 6 above to tokenize the input line.

   Note that your program must consist of at least three files: the main program (modified

from Exercise 1 above), and `token.h` and `token.c` from Exercise 2. You must write a makefile for building your program.

Unix Programming Exercises

1. **Exploring signal related header files**

   Explore the standard head files to find out all signals in your Linux system, including the name of the signals and their numeric numbers.

   Note the default location for all header files is directory `/usr/include`. Hence the path of the header file `<signal.h>` is `/usr/include/signal.h`.

   You should start your search from the header file `<signal.h>`. Depending on the operating system you are using, you may not necessarily find the signal definitions in file `<signal.h>`. However the file should provide pointers, ie the `include` directives, leading to other header files where the definitions are given.

   For instance, on *eris.ad.murdoch.edu.au*, by searching the string `include` in file:

   ```
   /usr/include/signal.h
   ```

   we quickly find the following `include` directive:

   ```
   #include <bits/signum.h>
   ```

   Then by viewing the file

   ```
   /usr/include/bits/signum.h
   ```

   we will find the signal definitions.

   Search string in `vi`: under the command mode, type `/include` to search for string `include`.

2. **Killing a process with Ctrl-C and Ctrl-**

   Run a program that lasts at least a few minutes (such as command line `sleep 100` or `cat`) on the foreground. Try to terminate the program by typing Ctrl-C or Ctrl-\. Explain what does Ctrl-C and Ctrl-\ do to the foreground processes.

3. **How to kill an editor?**

   Can you kill a running editor such as vi or pico with Ctrl-C or Ctrl-\? Try it.

   Explain your observations.

   How would you kill an editor then?

4. **Ignoring a signal**

   Write a program that would ignore SIGINT and SIGQUIT. If a program enters an infinite loop, how do you kill that program?

Note you may use the `signal` function in this exercise.

5. **Catching a signal**

   Modify the program from Exercise 4 above so that it catches SIGINT and SIGQUIT rather than ignoring them. Test your program to see whether it behaves as you expected.

   Note you may still use `signal` function in this exercise.

6. **Implementing the command `kill`**

   Write a program named `mykill` that does what the command `kill` does. Don't use `kill` command in your program.

7. **Use `sigaction` call**

   Modify the program from Exercise 6 above. Replace the `signal` function with the `sigaction` function in the new program.

8. **Catching different signals**

   Write a program that will

   a. create a child process. The child process just sleeps for 10 seconds before it terminates;
   b. set its alarm clock to 20 seconds (in the parent only);
   c. enters an infinite loop. In each loop, it prints one line and then sleep for one second.

   However when it (the parent) receives a signal it will respond in the following ways depending on the signal it received:

   - SIGCHLD: reports that the child has terminated and prints the child's pid;
   - SIGALRM: reports that alarm clock has expired.
   - SIGINT: reports SIGINT is received and prints its numeric value;
   - any other signal: performs its default action.

   Note this program must be written using reliable signal system calls (ie do not use the `signal` function)

   Test your program by sending it different signals.

9. **Blocking signals**

   Write a program that interacts with the user through the terminal (eg the program invites the user to type something from the keyboard and it prints a response based on what the user types). The interaction continues until the program is killed by Ctrl-C or Ctrl-\. However there is a twist: the user must be able to kill the program by typing Ctrl-C or Ctrl-\, but not before the program has run for at least 30 seconds. For example, if tbe user types Ctrl-C after the program has run for just 10 seconds, tbe program would not die straight away. It would continue its interactions with the user normally for another 20 seconds. Then it dies.

The Weekly Lab

Submit the following exercises:

1. C Programming Exercise 3.
2. Unix Programming Exercise 5.
3. Unix Programming Exercise 8.
4. Unix Programming Exercise 9.

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Sunday, 17 March 2024

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Sunday, 17-Mar-2024 10:09:34 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only